# RIVYLS

*Draft Programs. Not Players.*

---

## PREMIUM FEATURES
### SPECIFICATION & IMPLEMENTATION PLAN

Technical Reference for Platform Development
February 2026

# Table of Contents

1. Architecture Principle: Build for Premium from Day One3
2. Tier Structure Overview4
3. Free Tier: Complete Feature Specification5
4. Premium User Features: Competitive Edge7
5. Premium Commissioner Features: League Management10
6. Premium Content Layer: Insider Tier13
7. Database Schema Implications14
8. Feature Flag Architecture16
9. Implementation Phases & Build Order17
10. Founding Commissioner Program: Technical Requirements19

# 1. Architecture Principle: Build for Premium from Day One

The most expensive mistake in freemium platforms is retrofitting premium features onto a system that wasn't designed for them. Even though premium features won't ship until Year 2, the database schema, API layer, and component architecture must account for them from the initial build.

**CORE PRINCIPLE FOR CLAUDE CODE**

Every feature in this document falls into one of four categories. When building the Year 1 free tier, the developer must:

**1. BUILD NOW:** Features shipping in Year 1 free tier. Full implementation.

**2. SCHEMA NOW, BUILD LATER:** Database tables, columns, and relationships must exist now. UI and business logic built in Year 2.

**3. FLAG NOW, BUILD LATER:** Feature flag infrastructure must be in place. Feature gated behind a flag that returns false for all users until Year 2.

**4. PLAN ONLY:** Year 3+ feature. No code needed now, but avoid architectural decisions that would block it.

**What this means in practice:** The users table needs a 'tier' column now, even though every user will be 'free' in Year 1. The leagues table needs a 'scoring_preset' column and a 'custom_scoring' JSONB column now, even though custom scoring is a Year 2 commissioner feature. API endpoints should check user tier before returning data, even if the check always passes today. This costs almost nothing to implement upfront and saves weeks of refactoring later.

# 2. Tier Structure Overview

| Tier | Who | What They Get | Price | Ships |
|---|---|---|---|---|
| **Free** | All users, forever | Complete league experience: draft, scoring, transactions, leaderboard | $0 | Year 1 |
| **Rivyls Pro (User)** | Individual users | Pre-draft tools, analytics dashboard, transaction intelligence, advanced notifications, premium content | $4.99/mo or $29.99/yr | Year 2 |
| **Rivyls Pro (Commissioner)** | League commissioners | Custom scoring, flexible league structures, commissioner dashboard, draft customization, league history | Included with Pro subscription | Year 2 |
| **Founding Commissioner** | Early commissioners (Year 1) | All Pro features free for life when they launch | $0 (promotional) | Year 1 grant, Year 2 activation |

**The design test for every feature:** "Would a casual user miss this?" If yes, it belongs in Free. If only a power user or competitive player cares, it belongs in Pro. Free must be a complete, enjoyable experience that never feels crippled. Nobody converts to premium from frustration — they convert because they're already hooked and want to go deeper.

# 3. Free Tier: Complete Feature Specification

**Implementation:** BUILD NOW — All features in this section ship in Year 1.

## Authentication

| ID | Feature | Implementation | Schema / Notes |
|---|---|---|---|
| **FR-AUTH-01** | User registration via email, Google, Apple sign-in | *Supabase Auth* | users table with id, email, display_name, avatar_url, tier (default: 'free'), created_at, last_login |
| **FR-AUTH-02** | User profile (display name, avatar) | *Supabase Auth + profiles table* | profiles table linked to auth.users |

## League Creation

| ID | Feature | Implementation | Schema / Notes |
|---|---|---|---|
| **FR-LEAGUE-01** | Commissioner creates league with name and settings | *Supabase RLS* | leagues table: id, name, commissioner_id, sport (default: 'cfb'), season_year, status (draft/active/complete), scoring_preset, custom_scoring (JSONB, null for free), settings (JSONB), created_at |
| **FR-LEAGUE-02** | Shareable invite link generation | *Unique tokens* | league_invites table: id, league_id, token, expires_at, max_uses, current_uses |
| **FR-LEAGUE-03** | Join league via invite link | *Token validation* | league_members table: id, league_id, user_id, team_name, role (commissioner/member), joined_at |

## League Settings

| ID | Feature | Implementation | Schema / Notes |
|---|---|---|---|
| **FR-LEAGUE-04** | Preset scoring templates (Standard, Aggressive, Conservative) | *JSONB presets* | scoring_presets table or hardcoded constants. Free users select from presets only. Schema supports custom_scoring JSONB for Pro. |

## Draft

| ID | Feature | Implementation | Schema / Notes |
|---|---|---|---|
| **FR-DRAFT-01** | Snake draft with configurable timer | *Real-time via Supabase subscriptions* | drafts table: id, league_id, status, current_pick, draft_order (JSONB), timer_seconds. draft_picks table: id, draft_id, pick_number, user_id, school_id, picked_at |

| | | | |
|---|---|---|---|
| **FR-DRAFT-02** | Real-time draft board showing all picks | *Supabase real-time* | Subscription on draft_picks table filtered by draft_id |
| **FR-DRAFT-03** | School eligibility tracking during draft | *Server-side validation* | school_selections view: counts per school across league. Enforce max_times_school_allowed. |

## Roster Mgmt

| ID | Feature | Implementation | Schema / Notes |
|---|---|---|---|
| **FR-ROSTER-01** | View current roster with program details | *API endpoint* | rosters table: id, league_id, user_id, school_id, week_added, week_dropped (null if active), slot_number |
| **FR-ROSTER-02** | Add/drop transactions with validation | *Server-side validation* | transactions table: id, league_id, user_id, week, dropped_school_id, added_school_id, created_at. Validate: deadline, counter, eligibility. |
| **FR-ROSTER-03** | Transaction counter and deadline enforcement | *Computed from transactions* | league_settings stores max_transactions, transaction_deadline per week |

## Scoring

| ID | Feature | Implementation | Schema / Notes |
|---|---|---|---|
| **FR-SCORE-01** | Automated weekly scoring via ESPN API | *Cron job / scheduled function* | scores table: id, school_id, week, season_year, points, point_breakdown (JSONB). ESPN data cached in games table. |
| **FR-SCORE-02** | Live scoring updates during games | *Polling or real-time* | live_games table for active games. Supabase real-time subscriptions push updates to clients. |

## Standings

| ID | Feature | Implementation | Schema / Notes |
|---|---|---|---|
| **FR-STAND-01** | League leaderboard with total and weekly points | *Computed view* | league_standings view: aggregates scores by user across their roster for each week. Total, weekly breakdown. |
| **FR-STAND-02** | Individual team detail page (my roster, my scores) | *API endpoint* | Joins rosters + scores for user's active programs per week |

## Notifications

| ID | Feature | Implementation | Schema / Notes |
|---|---|---|---|
| **FR-NOTIF-01** | Basic email notifications (draft reminder, scoring updates, transaction window) | *Email service (Resend, SendGrid)* | notification_preferences table: user_id, email_scoring (bool), email_transactions (bool), email_draft (bool) |

## UI

| ID | Feature | Implementation | Schema / Notes |
|---|---|---|---|

| | | | |
|---|---|---|---|
| **FR-UI-01** | Mobile-responsive web interface | *Next.js + Tailwind* | No specific schema. Responsive breakpoints for phone, tablet, desktop. |

# 4. Premium User Features: Competitive Edge

**Implementation:** SCHEMA NOW, BUILD LATER — Database structures and feature flags in Year 1. Full UI and logic in Year 2.

### Pre-Draft Intelligence

| ID | Feature | Description | Schema / Notes | Phase |
|---|---|---|---|---|

### Live Analytics

| ID | Feature | Description | Schema / Notes | Phase |
|---|---|---|---|---|

### Transaction Intelligence

| ID | Feature | Description | Schema / Notes | Phase |
|---|---|---|---|---|

### Advanced Notifications

| ID | Feature | Description | Schema / Notes | Phase |
|---|---|---|---|---|

### Pre-Draft Intelligence

| ID | Feature | Description | Schema / Notes | Phase |
|---|---|---|---|---|
| **PRO-DRAFT-01** | **Program Power Rankings** | Composite score for each FBS program based on returning production, SOS, coaching stability, recruiting, preseason polls | *program_analytics table: school_id, season_year, power_score, sos_rank, returning_production_pct, recruiting_rank, preseason_rank, composite_score* | **SCHEMA NOW** |
| **PRO-DRAFT-02** | **Schedule Difficulty Analyzer** | Per-program schedule strength broken down by week, showing favorable/brutal stretches | *Derived from games table (opponent rankings, home/away). Computed view.* | **SCHEMA NOW** |
| **PRO-DRAFT-03** | **Conference Depth Analysis** | Conference-level power rankings showing top-heavy vs deep conferences for draft strategy | *Aggregated from program_analytics. conference_analysis view.* | **PLAN ONLY** |
| **PRO-DRAFT-04** | **Mock Draft Simulator** | Practice drafts against AI opponents with configurable difficulty | *mock_drafts table: id, user_id, settings (JSONB), results (JSONB), created_at. AI logic via server-side function.* | **PLAN ONLY** |

### Live Analytics

| ID | Feature | Description | Schema / Notes | Phase |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| PRO-ANALYTICS-01 | **Points Trend Chart** | Week-over-week points trend for each program on user's roster with visual graph | *Derived from scores table. No new schema needed — but API endpoint must be gated behind tier check.* | FLAG NOW |
| PRO-ANALYTICS-02 | **"What If" Transaction Calculator** | Simulate add/drop moves showing projected impact on standings | *Requires: current standings + program projections. transaction_simulations table or computed on-the-fly.* | SCHEMA NOW |
| PRO-ANALYTICS-03 | **Roster Optimization Suggestions** | AI-driven recommendations: "Your weakest program is X, available programs Y and Z are projected higher" | *Requires program_analytics + roster data. Computed server-side. recommendation_cache table.* | PLAN ONLY |
| PRO-ANALYTICS-04 | **Head-to-Head Comparison Tool** | Compare your roster against any other team: where you're winning/losing by program and week | *Derived from scores + rosters. No new schema — API endpoint gated.* | FLAG NOW |

## Transaction Intelligence

| ID | Feature | Description | Schema / Notes | Phase |
|---|---|---|---|---|
| PRO-TXN-01 | **Waiver Wire Analysis** | Trending available programs ranked by recent performance and upcoming schedule | *program_trends table: school_id, week, trend_score, recent_points_avg, upcoming_sos. Updated weekly via cron.* | SCHEMA NOW |
| PRO-TXN-02 | **Schedule Comparison (Add vs Current)** | Side-by-side comparison of a potential add's upcoming schedule vs your current program | *Derived from games table. No new schema — gated API endpoint.* | FLAG NOW |
| PRO-TXN-03 | **Watchlist with Alerts** | User can "watch" programs and receive alerts on availability changes or ranking moves | *watchlists table: id, user_id, school_id, created_at. Alert triggers via real-time subscriptions or cron.* | SCHEMA NOW |
| PRO-TXN-04 | **Historical Performance Data** | How programs perform in late season (Weeks 7-17) vs early season historically | *historical_performance table: school_id, season_year, early_season_ppg, late_season_ppg, bowl_performance. Populated from prior seasons.* | PLAN ONLY |

## Advanced Notifications

| ID | Feature | Description | Schema / Notes | Phase |
|---|---|---|---|---|
| PRO-NOTIF-01 | **Real-Time Game Alerts for Roster Programs** | Push/email alerts: "Alabama just beat a Top 10 team — you earned 3 bonus points" | *notification_queue table: id, user_id, type, title, body, data (JSONB), sent_at. Triggered by scoring engine.* | SCHEMA NOW |

| | | | | |
|---|---|---|---|---|
| **PRO-NOTIF-02** | **Personalized Weekly Preview** | Email with matchup analysis specific to user's roster programs for the upcoming week | *Generated by cron. Uses games + roster data. Template system.* | **PLAN ONLY** |
| **PRO-NOTIF-03** | **Transaction Recommendations** | Push alert when transaction window opens with suggested moves | *Requires PRO-TXN-01 waiver wire data. Triggered on transaction window open.* | **PLAN ONLY** |

# 5. Premium Commissioner Features: League Management

**Implementation:** SCHEMA NOW, BUILD LATER — These features map directly to the existing Google Sheets MVP's Settings sheet configuration. The database schema must support these values from Day 1, even though the UI to edit them is Year 2.

## Custom Scoring

| ID | Feature | Description | Schema / Notes | Phase |
|---|---|---|---|---|
| COMM-SCORE-01 | **Full Custom Scoring Rules** | Commissioner sets individual point values for all scoring categories (wins, conference games, ranked opponents, shutouts, bowl, playoff, Heisman, championship) | *leagues.custom_scoring JSONB column. Schema: { win: 1, conference_game: 1, over_50: 1, shutout: 1, ranked_25: 1, ranked_10: 2, conference_win: 10, loss: 0, conference_game_loss: 0, over_50_loss: 0, shutout_loss: 0, ranked_25_loss: 0, ranked_10_loss: 0, conference_loss: 0, heisman: 10, bowl_appearance: 5, playoff_first: 5, playoff_quarter: 5, playoff_semi: 5, championship_win: 20, championship_loss: 5 }* | SCHEMA NOW |
| COMM-SCORE-02 | **Loss Point Penalties** | Optional negative scoring for losses (configurable per loss type) | *Included in custom_scoring JSONB above. Free tier presets have loss values at 0.* | SCHEMA NOW |
| COMM-SCORE-03 | **Weekly High Points Prize Config** | Enable/disable weekly high points awards, set number of weeks, tie-breaking rules | *leagues.settings JSONB: { high_points_enabled: bool, high_points_weeks: int, ties_split: bool }* | SCHEMA NOW |

## Flexible Structure

| ID | Feature | Description | Schema / Notes | Phase |
|---|---|---|---|---|
| COMM-STRUCT-01 | **Variable Team Count (4-20)** | Commissioner sets number of teams in league | *leagues.settings.team_count. Free tier: fixed at 8 or 10. Pro: 4-20.* | SCHEMA NOW |
| COMM-STRUCT-02 | **Variable Roster Size (5-10 programs)** | Commissioner sets how many programs each team drafts | *leagues.settings.roster_size. Free: fixed at 7. Pro: 5-10.* | SCHEMA NOW |

| | | | | |
|---|---|---|---|---|
| COMM-STRUCT-03 | **Adjustable School Selection Limits** | How many times one program can be drafted across teams | *leagues.settings.max_school_selections. Free: default 3. Pro: 1-5.* | SCHEMA NOW |
| COMM-STRUCT-04 | **Adjustable Transaction Limits** | Max add/drops per team per season | *leagues.settings.max_transactions. Free: default 50. Pro: 1-unlimited.* | SCHEMA NOW |
| COMM-STRUCT-05 | **Trade Functionality** | Allow trades between teams (not just add/drop from free agents) | *trades table: id, league_id, proposing_user_id, receiving_user_id, proposed_schools (JSONB), requested_schools (JSONB), status (pending/accepted/rejected/expired), created_at, resolved_at* | SCHEMA NOW |
| COMM-STRUCT-06 | **Auction Draft Mode** | Alternative to snake draft: each program is nominated and bid on | *drafts.draft_type enum: 'snake', 'auction', 'linear'. auction_bids table: id, draft_id, school_id, user_id, bid_amount, is_winning.* | PLAN ONLY |

## Commissioner Dashboard

| ID | Feature | Description | Schema / Notes | Phase |
|---|---|---|---|---|
| COMM-DASH-01 | **League Activity Overview** | At-a-glance view: active users, last login per member, transaction counts, engagement metrics | *Derived from league_members + user activity logs. activity_log table: user_id, league_id, action, timestamp.* | SCHEMA NOW |
| COMM-DASH-02 | **Commissioner Roster Override** | Manual roster adjustments for dispute resolution | *transactions table needs a 'commissioner_override' boolean and 'override_reason' text field.* | SCHEMA NOW |
| COMM-DASH-03 | **League Announcements** | Commissioner sends messages to all league members | *league_announcements table: id, league_id, commissioner_id, title, body, created_at, pinned (bool).* | SCHEMA NOW |
| COMM-DASH-04 | **Prize Calculation Tools** | Automated prize pool and payout calculations based on entry fees and payout structure | *leagues.settings: { entry_fee, prize_structure: { first: pct, second: pct, third: pct }, high_points_amount }. Computed view for payouts.* | SCHEMA NOW |

## Draft Customization

| ID | Feature | Description | Schema / Notes | Phase |
|---|---|---|---|---|
| COMM-DRAFTC-01 | **Draft Scheduling with Calendar Invites** | Commissioner sets draft date/time, members receive calendar invites | *drafts.scheduled_at timestamp. Integration with email/calendar API.* | FLAG NOW |
| COMM-DRAFTC-02 | **Custom Timer per Round** | Different time limits for early vs late draft rounds | *drafts.timer_config JSONB: { default: 60, round_overrides: { 1: 120, 8: 30 } }* | SCHEMA NOW |

| | | | | |
|---|---|---|---|---|
| COMM-DRAFTC-03 | **Draft Lobby Chat** | In-draft chat room for league members during the draft | *draft_messages table: id, draft_id, user_id, message, created_at. Real-time via Supabase.* | PLAN ONLY |
| COMM-DRAFTC-04 | **Pause/Resume Draft** | Commissioner can pause draft for technical issues | *drafts.status enum includes 'paused'. drafts.paused_at, drafts.resumed_at timestamps.* | SCHEMA NOW |

## League History

| ID | Feature | Description | Schema / Notes | Phase |
|---|---|---|---|---|
| COMM-HIST-01 | **Season Archives** | Persistent season-by-season standings, results, and draft history | *league_seasons table: id, league_id, season_year, final_standings (JSONB), champion_user_id, archived_at. draft_picks persist across seasons.* | SCHEMA NOW |
| COMM-HIST-02 | **All-Time Records** | Cross-season stats: total championships, best seasons, head-to-head records | *Computed from league_seasons. all_time_records view.* | PLAN ONLY |
| COMM-HIST-03 | **Shareable League History Page** | Public URL showing league's multi-year history | *league.public_slug (unique string). Public API endpoint for read-only history.* | PLAN ONLY |

# 6. Premium Content Layer: Insider Tier

**Implementation:** PLAN ONLY — Year 3+ feature. No code needed in Year 1, but content delivery infrastructure should be considered when choosing a CMS or blog platform.

| ID | Feature | Description | Delivery |
|---|---|---|---|
| **CONTENT-01** | **Preseason Draft Guides** | Program-level draft rankings and analysis from a Rivyls-specific perspective | Blog/CMS with access gating by user tier |
| **CONTENT-02** | **Sleeper Programs Analysis** | Undervalued programs that casual drafters overlook | Seasonal content, published pre-draft |
| **CONTENT-03** | **Weekly Add/Drop Column** | In-season analysis of which available programs are trending up/down | Weekly content tied to transaction windows |
| **CONTENT-04** | **Playoff Picture Impact Analysis** | How CFP implications affect program values for Rivyls scoring | Late-season content (Weeks 10+) |
| **CONTENT-05** | **Personalized Season Review** | Per-user "year in review": best picks, worst picks, comparison to ideal team | Generated from scores + rosters data. Shareable graphics. |
| **CONTENT-06** | **Shareable Draft Results Card** | Visual card showing user's draft results for social media sharing | Generated image/HTML. Social sharing meta tags. |

# 7. Database Schema Implications

**Critical:** The following tables and columns MUST be included in the Year 1 database schema even though many won't be populated until Year 2. This prevents costly migrations and data loss later.

## Tables to Create in Year 1 (Even if Empty Until Year 2)

| Table | Column / Structure | Year 1 Action | Why Now |
|---|---|---|---|
| **users / profiles** | tier column (enum: 'free', 'pro', 'founding_commissioner') | **REQUIRED NOW** | Every API endpoint checks this value |
| **leagues** | custom_scoring JSONB column | **REQUIRED NOW** | Null for free-tier leagues; populated for Pro commissioners in Year 2 |
| **leagues** | settings JSONB with all configurable fields | **REQUIRED NOW** | Free leagues use defaults; Pro leagues allow overrides |
| **drafts** | draft_type enum ('snake', 'auction', 'linear') | **REQUIRED NOW** | Year 1 only supports 'snake'; enum prevents migration later |
| **drafts** | timer_config JSONB | **REQUIRED NOW** | Year 1 uses flat timer; Pro allows per-round config |
| **drafts** | paused_at, resumed_at timestamps | **REQUIRED NOW** | Null in Year 1; used by commissioner pause/resume in Year 2 |
| **transactions** | commissioner_override bool, override_reason text | **REQUIRED NOW** | Default false; enables commissioner dispute resolution in Year 2 |
| **trades** | Full table structure | **CREATE NOW (EMPTY)** | No Year 1 UI; schema must exist for Year 2 trades feature |
| **program_analytics** | Full table structure | **CREATE NOW (EMPTY)** | Populated in Year 2 for pre-draft intelligence features |
| **program_trends** | Full table structure | **CREATE NOW (EMPTY)** | Populated in Year 2 for waiver wire analysis |
| **watchlists** | Full table structure | **CREATE NOW (EMPTY)** | Year 2 feature; simple schema, no cost to create now |
| **notification_queue** | Full table with type, data JSONB | **REQUIRED NOW** | Year 1 uses for basic emails; Year 2 expands to real-time alerts |
| **notification_preferences** | Columns for all notification types | **REQUIRED NOW** | Year 1 toggles basic email; Year 2 adds push, SMS, advanced alerts |
| **league_announcements** | Full table structure | **CREATE NOW (EMPTY)** | Year 2 commissioner feature |
| **activity_log** | user_id, league_id, action, timestamp | **CREATE NOW** | Start logging in Year 1; critical for commissioner dashboard and analytics |

| | | | |
|---|---|---|---|
| **league_seasons** | Full table structure | **CREATE NOW (EMPTY)** | Populated at end of Year 1 season; enables league history in Year 2 |
| **draft_messages** | Full table structure | **CREATE NOW (EMPTY)** | Year 3 draft lobby chat |

# 8. Feature Flag Architecture

Every premium feature must be gated behind a feature flag that can be toggled per-user or globally. This enables: gradual rollout of premium features, A/B testing different feature sets, the Founding Commissioner program (grant Pro features to specific users), and instant kill-switch if a feature causes issues.

## Recommended Implementation

**Simple approach (Year 1):** A feature_flags table with columns: id, flag_name, is_global (bool), global_value (bool), created_at. A user_feature_flags table with: user_id, flag_name, value (bool), granted_at, expires_at (nullable). The check logic: if user_feature_flags has an entry for this user+flag, use that value. Otherwise, use the global_value. Otherwise, default false.

### API middleware pattern:

*Every API endpoint that serves premium data should include a middleware check:*

**PSEUDOCODE: Feature Gate Middleware**

```
function requireFeature(flagName) {
  const user = getCurrentUser();
  const hasAccess = checkFeatureFlag(user.id, flagName);
  if (!hasAccess) return { error: 'Premium feature', upgrade_url: '/pro' };
  return next();
}
```

### Flags to create in Year 1 (all default false):

| Flag Name | Controls |
|---|---|
| **pro_analytics** | Unlocks analytics dashboard, trend charts, head-to-head comparisons |
| **pro_draft_intelligence** | Unlocks program power rankings, schedule analyzer, mock drafts |
| **pro_transaction_intelligence** | Unlocks waiver wire analysis, watchlist alerts, schedule comparison |
| **pro_notifications_advanced** | Unlocks real-time game alerts, personalized previews |
| **pro_commissioner_scoring** | Unlocks custom scoring editor for commissioner |
| **pro_commissioner_structure** | Unlocks flexible team count, roster size, trades |
| **pro_commissioner_dashboard** | Unlocks activity overview, roster override, announcements |
| **pro_commissioner_draft** | Unlocks draft scheduling, custom timers, pause/resume |

| | |
|---|---|
| **pro_league_history** | Unlocks season archives, all-time records |
| **pro_content** | Unlocks premium content (draft guides, weekly analysis) |

# 9. Implementation Phases & Build Order

---

### Year 1(2026 Season) — BUILD NOW

- All Free Tier features (FR-* IDs)
- Database schema for ALL tables (including empty Year 2 tables)
- Feature flag infrastructure (feature_flags + user_feature_flags tables)
- User tier column on profiles table (default: 'free')
- Activity logging (activity_log table — start collecting data immediately)
- Notification queue and preferences tables (basic email in Year 1)
- League settings JSONB with defaults (schema supports Pro customization)
- Scoring engine reads from custom_scoring JSONB (uses preset values for free tier)

### Year 2 Q1Pre-Season(Mar-Jul 2027) — COMMISSIONER TOOLS

- COMM-SCORE-01: Custom scoring UI for Pro commissioners
- COMM-STRUCT-01 through 04: Flexible league structure settings
- COMM-DASH-01 through 04: Commissioner dashboard
- COMM-DRAFTC-01, 02, 04: Draft scheduling, custom timers, pause/resume
- COMM-HIST-01: Season archives (auto-populated from Year 1 data)
- Stripe integration for Pro subscriptions ($4.99/mo or $29.99/yr)
- Founding Commissioner flag activation (flip pro_commissioner_* flags for qualifying users)

### Year 2 Q2In-Season(Aug-Jan 2027-28) — USER ANALYTICS

- PRO-ANALYTICS-01: Points trend charts
- PRO-ANALYTICS-04: Head-to-head comparison
- PRO-TXN-01: Waiver wire analysis (populate program_trends table)
- PRO-TXN-02: Schedule comparison tool
- PRO-TXN-03: Watchlist with alerts
- PRO-NOTIF-01: Real-time game alerts
- Activate pro_analytics and pro_transaction_intelligence flags for Pro users

### Year 2 Q3Off-Season(Feb-Jul 2028) — DRAFT INTELLIGENCE

- PRO-DRAFT-01: Program power rankings (populate program_analytics table)
- PRO-DRAFT-02: Schedule difficulty analyzer
- COMM-STRUCT-05: Trade functionality
- CONTENT-01, 02: Preseason draft guides and sleeper analysis
- CONTENT-05: Personalized season review (generated from Year 2 data)

### Year 3+(2028-29) — ADVANCED FEATURES

- PRO-DRAFT-03: Conference depth analysis
- PRO-DRAFT-04: Mock draft simulator with AI opponents
- PRO-ANALYTICS-02, 03: What-if calculator, roster optimization AI
- COMM-STRUCT-06: Auction draft mode
- COMM-DRAFTC-03: Draft lobby chat
- COMM-HIST-02, 03: All-time records, public league history page

- CONTENT-03, 04: Weekly columns, playoff impact analysis
- Native mobile app (iOS/Android)

# 10. Founding Commissioner Program: Technical Requirements

Commissioners who create and fill a league during the Year 1 (2026) season receive all Pro features free for life when premium launches in Year 2. This is the primary incentive for early commissioner recruitment.

## Qualification Criteria

**A commissioner earns Founding Commissioner status by meeting ALL of these conditions:**

**1.** Created a league during the 2026 season (leagues.season_year = 2026)

**2.** League reached minimum viable size (e.g., 6+ members joined)

**3.** League completed a draft (drafts.status = 'complete')

**4.** League remained active through at least Week 12 (majority of members still logging in)

## Technical Implementation

**1. Tier assignment:** At the end of the 2026 season, run a migration that sets users.tier = 'founding_commissioner' for all qualifying commissioners. This tier grants all Pro features without requiring a subscription.

**2. Feature flag grants:** Insert rows into user_feature_flags for each qualifying commissioner, setting all pro_commissioner_* flags to true with expires_at = null (lifetime access).

**3. Visual badge:** profiles table should include a badges JSONB column. Founding commissioners receive { founding_commissioner: { granted: '2027-02-01', display: true } }. This badge appears on their profile and in league views.

**4. Communication:** Automated email to all qualifying commissioners announcing their Founding Commissioner status and what it includes. This is a retention and advocacy moment.

### SUMMARY FOR CLAUDE CODE

When building the Year 1 platform, the developer should:

**1.** Include ALL database tables listed in Section 7, even if empty

**2.** Add tier column to users/profiles (enum: free, pro, founding_commissioner)

**3.** Create feature_flags and user_feature_flags tables

**4.** Use leagues.settings JSONB and leagues.custom_scoring JSONB from Day 1

**5.** Make the scoring engine read from custom_scoring JSONB (hardcode preset values for free)

**6.** Start logging to activity_log table immediately

**7.** Gate API endpoints with tier checks even if all users are free today

**8.** Use feature IDs (FR-*, PRO-*, COMM-*, CONTENT-*) in code comments for traceability

---

**END OF PREMIUM FEATURES SPECIFICATION**

*Rivyls — Draft Programs. Not Players.*